

Rylan Wade

CSCI-546 Distributed Systems

Professor Seo Jin Park

4/21/2026

Lab 3: High-Performance Fault-Tolerant Key/Value Service

The KV server is built on top of Raft, a consensus algorithm designed to make a cluster of nodes agree on a shared, ordered log of operations. Raft works by electing a single leader at any given time. All client writes go through the leader, which appends the operation to its local log and replicates it to a majority of follower nodes. Once a majority acknowledges the entry, it is committed and will survive any future election and will eventually be applied to every node's state machine. If the leader crashes, followers hold a new election, and the cluster continues making progress as long as a majority of nodes remain alive. The guarantee that all nodes apply operations in the same order is what makes it safe to build a fault-tolerant key-value store on top of Raft. The KV layer treats Raft as a black box that guarantees that operations are applied in the order they are sent.

When a client issues a Put, Get, or Append RPC, the KV server's handler serializes the operation and passes it to the Raft layer via `raft_.propose`. If the node is not the current leader, `propose` returns immediately with a not-leader signal. If the node is the leader, `propose` appends the entry to the local Raft log, records the current log index, and wakes the background replication timer so the entry is sent to followers without waiting for the next scheduled heartbeat. The Raft layer then replicates the entry via `AppendEntries` RPCs, and once a majority of nodes acknowledge it, the leader commits the entry and delivers it to the KV layer's `on_apply`.

The RPC handler waits for its committed entry using a map of futures and promises keyed by log index. After calling `propose`, a promise is registered at the assigned log index and the handler blocks on the corresponding future with a two-second timeout. When `on_apply` fires for that index, it sets the promise's value and unblocks the handler. The handler then verifies that the committed entry at that index actually belongs to its own operation by searching the command string for its client ID and sequence number.

Duplicate detection is implemented through RIFL, which prevents a retried request from being applied twice if its original reply is dropped. Each client carries a unique ID and a monotonically increasing sequence number, and the server maintains a map from client ID to the most recently completed operation. Deduplication is enforced at two levels: before the operation reaches Raft, the handler checks whether the sequence number has already been applied and returns the cached result immediately if so. Inside `on_apply`, the same check fires before touching the store, handling the case where a retry on a new leader causes the same operation to be committed a second time.

Figures 1 and 2 show the latency vs. throughput plots for 3 and 5 replica clusters with a 50% put ratio. Both clusters maintain stable P50 latency of around 1–2ms under light load, with all percentiles diverging sharply at high concurrency. The 3-replica cluster reaches a peak throughput of roughly 8,300 ops/sec compared to 7,000 ops/sec for 5 replicas. This gap is expected: with more replicas the leader must collect acknowledgments from a larger quorum before committing each entry, adding replication overhead on every write.

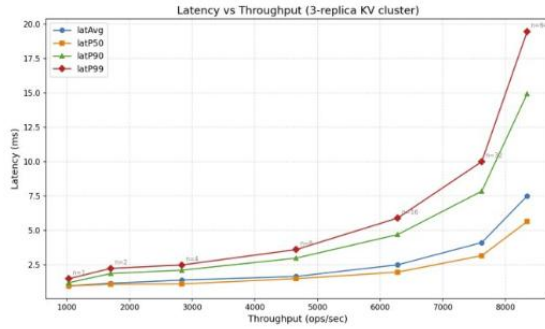


Figure 1: 3-replica cluster and 50% put ratio

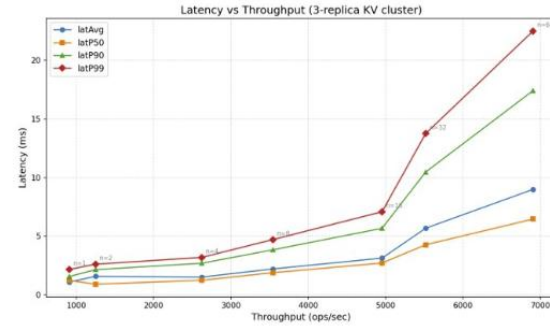


Figure 2: 5-replica cluster and 50% put ratio

Five optimizations were built to improve performance, applied incrementally on top of one another. Operations per second (ops/sec) measures peak throughput at 64 concurrent clients with a 50% put ratio, and P50 latency measures the median round-trip time for a single client. The table below shows the performance metrics after each stage of development.

Optimization Name	Reason	Ops/sec	P50 ms
Baseline	No optimizations	2179	2.19
Read lease	Get requests went through a full Raft round trip unnecessarily; a valid lease proves no other leader exists so reads can be served locally	4499	1.46
Parallelize AppendEntries across peers	Serial replication meant latency was sum of all peer round trips; parallelizing makes it max of peer round trips	6123	0.66
Reduce heartbeat interval to 2ms	Proposals that miss the immediate-replication wake-up waited too long; lower interval also enables implicit batching of burst proposals into one AppendEntries RPC	4293	1.44
Immediate replication on propose()	Proposals waited up to one full heartbeat interval before replicating; waking the timer thread on propose() eliminates this delay	6045	0.44
Downgrade logging to debug level	info-level logging on the hot path (timer loop, AppendEntries handler, send_to_peer_) caused synchronous I/O on every RPC	7252	0.92